

Derivations

Concepts of Programming Languages Lecture 6

Expressions

» Let-expressions

» If-Expressions

» **Functions**

» Application

Functions (Syntax Rule)

$$\langle \text{expr} \rangle ::= \text{fun } \langle \text{var} \rangle \rightarrow \langle \text{expr} \rangle$$

Functions (Syntax Rule)

$$\langle \text{expr} \rangle ::= \text{fun } \langle \text{var} \rangle \rightarrow \langle \text{expr} \rangle$$

If x is a valid variable name and e is a well-formed expression, then

$$\text{fun } x \rightarrow e$$

is a well-formed expression

Functions (Typing Rule)

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_1 \rightarrow \tau_2}$$

Functions (Typing Rule)

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_1 \rightarrow \tau_2} \text{ (fun)}$$

If e has type τ_2 in the context Γ with the variable declaration $(x : \tau_1)$ added, then

$\text{fun } x \rightarrow e$

is of type $\tau_1 \rightarrow \tau_2$ in the context Γ

Functions (Semantic Rule)

$(\text{fun } x \rightarrow e) \Downarrow \lambda x. e$

Functions (Semantic Rule)

$$\frac{}{\text{fun } x \rightarrow e \Downarrow \lambda x . e} \text{ (funEval)}$$

Under no premises, the expression

$\text{fun } x \rightarrow e$

evaluates to the *function value* $\lambda x . e$ (we'll talk more about function values later)



Expressions

» Let-expressions

» If-Expressions

» Functions

» **Application**

Application (Syntax Rule)

$$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle \langle \text{expr} \rangle$$

Application (Syntax Rule)

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle \langle \text{expr} \rangle$

If e_1 is a well-formed expression and e_2 is a well-formed expression, then $e_1 e_2$ is a well-formed expression

Application (Typing Rule)

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash \underline{e_1} \underline{e_2} : \tau}$$

If e_1 has type $\tau_2 \rightarrow \tau$ under the context Γ and e_2 is of type τ_2 under the context Γ , then $e_1 e_2$ is of type τ under the context Γ

Application (Typing Rule)

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau} \text{ (app)}$$

if (3 > 0) {
 return fun x → x + 1
}
else { return fun x → 2 * x }

If e_1 has type $\tau_2 \rightarrow \tau$ under the context Γ and e_2 is of type τ_2 under the context Γ , then $e_1 e_2$ is of type τ under the context Γ

if e_1 then e_2 else e_3

Application (Semantic Rule)

$$\begin{array}{c}
 \lambda x. \boxed{x+1} \quad \Downarrow 4 \quad \cancel{[4/x]x+1} \rightarrow 4+1 \\
 e_1 \Downarrow \lambda x. e \quad e_2 \Downarrow v_2 \quad [v_2/x]e \Downarrow v \\
 \hline
 \text{(appEval)}
 \end{array}$$

$e_1 \rightarrow$

 if $3 > 0$
 then e_1

 $\text{fun } x \rightarrow x+1$
 else e

 $\text{fun } x \rightarrow 2 * x$

1. e_1 evaluates to a function value $\lambda x. e$
2. e_2 evaluates to v_2
3. e with v_2 substituted for x evaluates to v

It follows that $e_1 e_2$ evaluates to v

Practice Problem

Recall the `=` operator in OCaml. Write down (to the best of your ability) the syntax, typing, and semantic rules for `=`

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle = \langle \text{expr} \rangle$

$$\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 = e_2 : \text{bool}} \quad (\text{eq})$$
$$\frac{e_1 \Downarrow v \quad e_2 \Downarrow v}{e_1 = e_2 \Downarrow \text{true}} \quad (\text{eq true}) \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 \neq v_2}{e_1 = e_2 \Downarrow \text{false}} \quad (\text{eq false})$$

Outline

- » Discuss derivations in general
- » See how to read and write derivations
- » Go through a couple examples

Recap

Let-Expressions

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau} \text{ (let)}$$

$$\frac{e_1 \Downarrow v_1 \quad [v_1/x]e_2 \Downarrow v}{\text{let } x = e_1 \text{ in } e_2 \Downarrow v} \text{ (letEval)}$$

If-Expressions

$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau} \text{(if)}$$

$$\frac{e_1 \Downarrow \text{true} \quad e_2 \Downarrow v_2}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v_2} \text{(ifEvalTrue)}$$

$$\frac{e_1 \Downarrow \text{false} \quad e_3 \Downarrow v_3}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v_3} \text{(ifEvalFalse)}$$

$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau}$$

Derivations

High Level

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau}$$

$\frac{}{\{\} \vdash 2 : \text{int}} \text{ (intLit)}$	$\frac{}{\{y : \text{int}\} \vdash y : \text{int}} \text{ (var)}$	$\frac{}{\{y : \text{int}\} \vdash y : \text{int}} \text{ (var)}$
$\frac{\{y : \text{int}\} \vdash y : \text{int} \quad \{y : \text{int}\} \vdash y : \text{int}}{\{y : \text{int}\} \vdash y + y : \text{int}} \text{ (intAdd)}$		
$\frac{\{\} \vdash 2 : \text{int} \quad \{y : \text{int}\} \vdash y + y : \text{int}}{\{\} \vdash \text{let } y = 2 \text{ in } y + y : \text{int}} \text{ (let)}$		

Derivations *prove* that a judgment holds w.r.t some rules

A **derivation** is a tree in which:

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}}$$

» each node is labeled with a judgment

» and judgment *follows* from the judgments at its children by an **inference rule**

$$(2+3)^2 = 2^2 + 3^2 + 2 \cdot 2 \cdot 3$$

$$(a+b)^2 = a^2 + b^2 + 2ab$$

Applying Rules

$$\frac{}{\Gamma \vdash [] : \tau \text{ list}} \text{ (nil)} \qquad \frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau \text{ list}}{\Gamma \vdash e_1 :: e_2 : \tau \text{ list}} \text{ (cons)}$$

$\{x : \text{int}\} \vdash x + 1 : \text{int}$	$\{x : \text{int}\} \vdash [] : \text{int list}$	(cons)
$\{x : \text{int}\} \vdash (x + 1) :: [] : \text{int list}$		

So far, we've used rules as ways of describing the behavior of a PL

When we build typing derivations, we *instantiate* the meta-variables in the rule at *particular* expressions, contexts, etc.

Building from the Ground Up

$\frac{}{\{x : \text{int}\} \vdash x : \text{int}} \text{ (var)}$	$\frac{}{\{x : \text{int}\} \vdash 1 : \text{int}} \text{ (intLit)}$	
$\frac{}{\{x : \text{int}\} \vdash x + 1 : \text{int}} \text{ (intAdd)}$	$\frac{}{\{x : \text{int}\} \vdash [] : \text{int list}} \text{ (nil)}$	
$\frac{\{x : \text{int}\} \vdash x + 1 : \text{int} \quad \{x : \text{int}\} \vdash [] : \text{int list}}{\{x : \text{int}\} \vdash (x + 1) :: [] : \text{int list}} \text{ (cons)}$		

But we can't *just* apply rules, because it's possible that the premises of a rule **also need to be demonstrated**

This is how we get our tree structure: we apply rules from the ground up

Axioms (When are we done?)

$\frac{}{\{x : \text{int}\} \vdash x : \text{int}} \text{ (var)}$	$\frac{}{\{x : \text{int}\} \vdash 1 : \text{int}} \text{ (intLit)}$	
$\frac{}{\{x : \text{int}\} \vdash x + 1 : \text{int}} \text{ (intAdd)}$	$\frac{}{\{x : \text{int}\} \vdash [] : \text{int list}} \text{ (nil)}$	
	$\frac{}{\{x : \text{int}\} \vdash (x + 1) :: [] : \text{int list}} \text{ (cons)}$	

The leaves of the tree are **axioms**, i.e., a rules with no premises

In our case, this will almost always be "literal" or "variable" rules

Integer Literals

(1)

$$\frac{n \text{ is an int lit}}{\Gamma \vdash n : \text{int}} \quad (\text{intLit})$$

(2)

$$\frac{n \text{ is an int lit}}{n \Downarrow n} \quad (\text{intLitEval})$$

~~$\frac{\Gamma \vdash e_1 \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \quad (\text{intLit})$~~

1. If n is an integer literal, then it is of type int in any context
2. If n is an integer literal, then it evaluates to the number it represents

A Note about Side Conditions

we don't write "1 is an integer literal"

$\frac{}{\{x : \text{int}\} \vdash x : \text{int}} \text{ (var)}$		$\frac{}{\{x : \text{int}\} \vdash 1 : \text{int}} \text{ (intLit)}$	
	$\frac{}{\{x : \text{int}\} \vdash x + 1 : \text{int}} \text{ (intAdd)}$		$\frac{}{\{x : \text{int}\} \vdash [] : \text{int list}} \text{ (nil)}$
		$\frac{}{\{x : \text{int}\} \vdash (x + 1) :: [] : \text{int list}} \text{ (cons)}$	

If a premise is a side-condition this *it is not included in the derivation*

Side conditions need to hold in order to apply the rule, but they don't appear in the derivation itself

We will always make side conditions clear

Float Literals

$$(1) \quad \frac{n \text{ is an float lit}}{\Gamma \vdash n : \text{float}} \quad (\text{floatLit})$$

$$(2) \quad \frac{n \text{ is an float lit}}{n \Downarrow n} \quad (\text{floatLitEval})$$

1. If n is an float literal, then it is of type float in any context
2. If n is an float literal, then it evaluates to the number it represents

Boolean Literals

$$(1) \quad \frac{}{\Gamma \vdash \text{true} : \text{bool}} \text{ (trueLit)}$$

$$(2) \quad \frac{}{\Gamma \vdash \text{false} : \text{bool}} \text{ (falseLit)}$$

$$(3) \quad \frac{}{\text{true} \Downarrow \text{true}} \text{ (trueLitEval)}$$

$$(4) \quad \frac{}{\text{false} \Downarrow \text{false}} \text{ (falseLitEval)}$$

1. `true` is of type `bool` in any context
2. `false` is of type `bool` in any context
3. `true` evaluates to the value $\top$
4. `false` evaluates to the value $\perp$

Variables

$$\frac{(\cancel{x} : \tau) \in \Gamma}{\Gamma \vdash \cancel{x} : \tau} \quad \begin{array}{c} \text{var} \\ \text{intLit} \end{array}$$

If v is declared to be of type τ in the context Γ , then v is of type τ in Γ

Variables cannot be evaluated (more on this when we talk about substitution and well-scopedness)

Back to the Example

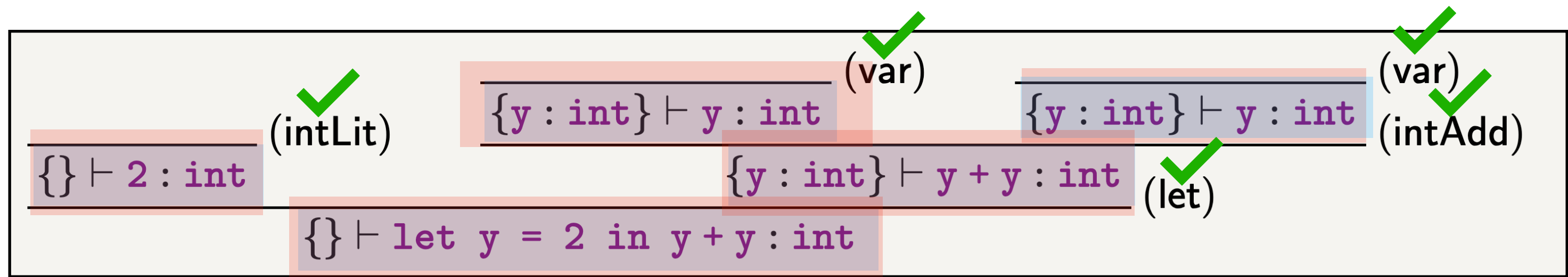
$$\frac{\frac{}{\{\} \vdash 2 : \text{int}} \text{(intLit)} \quad \frac{\frac{}{\{y : \text{int}\} \vdash y : \text{int}} \text{(var)} \quad \frac{\frac{}{\{y : \text{int}\} \vdash y : \text{int}} \text{(var)}}{\{y : \text{int}\} \vdash y + y : \text{int}} \text{(intAdd)}}{\{\} \vdash \text{let } y = 2 \text{ in } y + y : \text{int}} \text{(let)}$$

We need $\{\} \vdash 2 : \text{int}$ in order to proof that the bottom typing judgment holds

Now we know that this follows from the **intLit** rule, which says that 2 is always an int, *by fiat*

Okay, I know that was a
lot, let's take a step back

Derivations Encode Natural Language Arguments



$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \text{ (let)}$$

$$\frac{\text{n is an integer literal}}{\Gamma \vdash \text{n} : \text{int}} \text{ (intLit)}$$

The expression `let y = 2 in y + y` is an `int` because

» `2` is an `int` by fiat (and so `y` is being assigned to a well-typed

» and, assuming `y` is an `int`, `y + y` is an `int` because

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \text{ (addInt)}$$

- `y` is an `int` (by assumption)
- and so is `y` (by assumption)

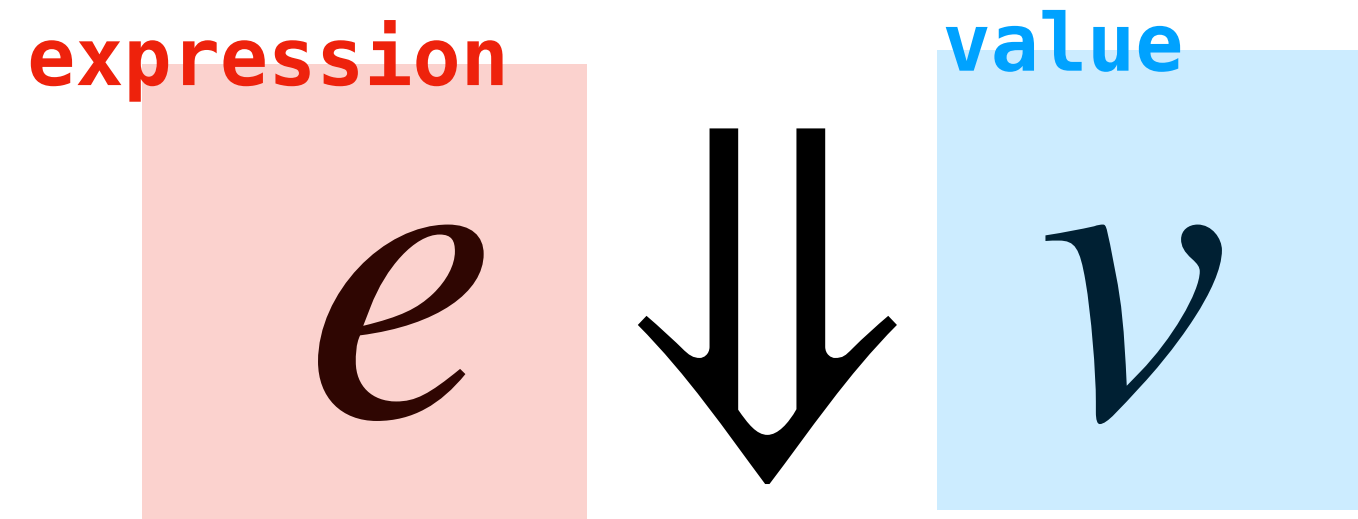
$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau} \text{ (var)}$$

and so integer-adding these two expressions (`y` and `y`) yields an `int`

and so assigning `y` to `2` in `y + y` yields an `int`

And all this works for
semantics judgements as well

Recall: Semantic Judgements



A **semantic judgment** is a compact way of representing the statement:

The expression e evaluates to the value v

A **semantic rule** is an inference rule with semantic judgments

Recall: Integer Addition Semantic Rule

$$\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 + v_2 = v}{e_1 + e_2 \Downarrow v} \text{ (evalInt)}$$

If e_1 evaluates to the (integer) v_1 and e_2 evaluates to the (integer) v_2 , and $v_1 + v_2 = v$, then $e_1 + e_2$ evaluates to the (integer) v

Semantic Derivations

$$\frac{\frac{}{\text{true} \Downarrow \text{true}} \text{ (trueEval)} \quad \frac{}{2 \Downarrow 2} \text{ (intEval)}}{\text{if true then 2 else 3} \Downarrow 2 \text{ (ifEval)}}$$

We can also write derivations to prove semantic judgments

The principle is the same, except that the judgments are semantic judgments instead of typing judgments

Examples

Example (Typing)

$$\frac{\underbrace{\vdash e_1 : \text{bool}}_{\text{(trueLit)}} \quad \underbrace{\vdash e_2 : \tau}_{\text{(intLit)}} \quad \underbrace{\vdash e_3 : \tau}_{\text{(intLit)}}}{\vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau} \text{(if)}$$

$$\frac{\underbrace{\{\} \vdash \text{true} : \text{bool}}_{\text{(trueLit)}} \quad \underbrace{\{\} \vdash 2 : \text{int}}_{\text{(intLit)}} \quad \underbrace{\{\} \vdash 5 : \text{int}}_{\text{(intLit)}}}{\{\} \vdash \text{if true then 2 else 5 : int}} \text{(if)}$$

Example (Evaluation)

$e_1 \Downarrow \text{true}$ $e_2 \Downarrow v_2$
if e_1 then e_2 else $e_3 \Downarrow v_2$

$\text{true} \Downarrow \text{true}$ (trueLitEval) $2 \Downarrow 2$ (intLitEval)

if true then 2 else 5 $\Downarrow$ 2 (ifTrueEval)

Example (Typing)

$\{\} \vdash 2 + 3 \triangleleft 4 : \text{bool}$

Example (Evaluation)

$2 + 3 \diamond 4 \Downarrow \text{true}$

Example (Evaluation)

$$\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2}{e_1 + e_2 \Downarrow v} \quad \text{let } \underline{x} = \underline{e_1} \text{ in } \underline{e_2} \Downarrow \underline{v}$$

Handwritten diagram illustrating the evaluation of the expression $2 + 2$ using a stack-based approach:

The diagram shows the state of the stack and the current expression being evaluated at different stages:

- Initial state: The stack contains the integer literal 2 . The expression being evaluated is 2 (labeled `(intLiteral)`).
- Push operation: The second 2 is pushed onto the stack. The stack now contains 2 and 2 . The expression being evaluated is $2 + 2$ (labeled `(intAddEval)`).
- Pop operation: The top element of the stack (2) is popped. The stack now contains only the bottom 2 . The expression being evaluated is 4 (labeled `(letEval)`).

The final result of the evaluation is 4 .

let $x = 2$ in $x + x$ $\Downarrow$ 4

Summary

Derivations are **tree-like proofs** that judgments hold with respect to a collection of inference rules

Derivations are **compact mathematical representations** of English language arguments

Learning to write derivations takes *practice*